

AI Inference & Hosting Infrastructure Benchmarks

Production-grade prompts from a single replica to multi-region failover,
ordered by industry difficulty, with mechanical grader contracts

Revision 2 expands the ladder through L9. L1-L6 remain engine-and-SLO fundamentals. L7 is the 2026 Kubernetes well-lit path (KServe LLMInferenceService, llm-d InferencePool, Gateway API Inference Extension EndpointPicker, DRA, Kueue gang scheduling). L8 is Slurm vs Kubernetes vs Slinky hybrids and SLA-driven graph deployments (Dynamo-style). L9 is hierarchical KV (Mooncake/LMCache/DualPath) and encode-prefill-decode on mixed accelerators. Later prompts assume vocabulary from earlier ones.

Standards vocabulary (use these names)

TTFT — time to first token. TPOT — time per output token (mean). ITL — inter-token latency histogram. Goodput — tokens that met the latency SLO, not raw decode tokens/s. KV pool — paged KV capacity in tokens or blocks. Prefix/radix cache — token-id-keyed tree of KV blocks. PD disaggregation — separate prefill and decode pools plus a KV handoff. Error budget / burn rate — SRE availability accounting. Ready vs live — Kubernetes probe split. Gateway — auth, schema, quota, canary; not the GPU scheduler.

How to run: one prompt per session or chained L1→L6 with a living INFRA.md. Grade artifacts, not prose. A missing metric name or a softened SLO is a fail. Section L6 is the industry-hard combined exam.

L1 Foundational serving

1. Single-Replica OpenAI-Compatible Server

Objective

Stand up one replica that speaks the de-facto industry API (POST /v1/chat/completions + /v1/models + /health) and emits the minimum production metrics.

Prompt to try

You are the on-call inference engineer for a new model endpoint. Produce a complete serving spec and runbook for ONE replica.

Constraints (do not weaken):

- API: OpenAI-compatible Chat Completions, streaming (SSE, data: ... then data: [DONE]) and non-streaming.
- Required headers: x-request-id (generated if absent), x-model.
- Health: /health/live (process up) and /health/ready (weights loaded, GPU context created). Ready must be 503 until first successful dummy generate.
- Metrics (Prometheus text):
 - llm_ttft_seconds (histogram)
 - llm_tpot_seconds (histogram) # time per output token
 - llm_requests_total{status,model}
 - llm_tokens_total{dir="in"|"out",model}
 - gpu_duty_cycle (0-1 gauge)
- Config: max_model_len, max_num_seqs, gpu_memory_utilization, dtype in {fp16,bf16,fp8}.
- Failure mode: if weights path is missing, ready stays 503 and metrics still scrape.

Stages:

1. Write openapi.yaml for the three routes + error envelope {error:{type,message,code}}.
2. Write the process lifecycle: load -> warm dummy prompt -> ready -> serve.
3. Write a k6 or vegeta-style load profile: 8 concurrent streams, 128-token prompts, 64-token max_tokens, 2 minutes.
4. State the acceptance SLO for this L1 box only: ready within 180s of process start on a single 80GB GPU; p95 TTFT < 2.0s for the load profile above; no 5xx on /health/live.

Artifacts: openapi.yaml, runbook.md, metrics.md, load_profile.json, slo.md.

Grader contract

Fail if streaming is described without SSE framing, if live and ready are the same probe, or if TTFT/TPOT are missing. Dummy-warm before ready is required. No multi-GPU claims at L1.

2. Continuous Batching and Queue Discipline

Objective

Specify iteration-level (continuous) batching with an explicit admission controller and a bounded waiting queue — the standard vLLM/TGI scheduler contract.

Prompt to try

Design the scheduler for a single GPU replica that does iteration-level batching.

Must define:

- Waiting queue Q and running set R. Capacity: max_num_seqs, max_num_batched_tokens, max_model_len.
- Admission: a request enters R only if remaining KV pages + new pages fit in the paged-KV pool.
- Preemption policy: choose one of {recompute, swap} and state when it fires (KV pool > watermark).
- Starvation bound: a waiting request of priority p is admitted within T_starve tokens of wall-clock or it is 429'd with Retry-After.
- Fairness: FCFS inside a priority class; two classes {interactive, batch}.

Stages:

1. Write the scheduler state machine (admit, step, preempt, finish).
2. Give a worked example: 4 requests with prefill lengths 32, 512, 2048, 128 arriving 0, 10, 20, 30 ms apart on a pool of 8192 tokens. Show Q, R, and KV occupancy after each step until all complete or one is rejected.
3. Define metrics: llm_scheduler_running, llm_scheduler_waiting, llm_kv_cache_usage_ratio, llm_preempt_total{reason}.
4. Prove (informal invariant): at every step, sum of reserved KV pages <= pool size.

Artifacts: scheduler.md, worked_example.md, metrics.md, invariant.md.

Grader contract

Worked example must keep KV occupancy <= 8192 at every row. Preempt policy named. Starvation bound numeric. FCFS-inside-class stated. Missing occupancy table = fail.

L2 Production SLOs and tenancy

3. SLO Pack: TTFT, TPOT, Goodput, Error Budget

Objective

Replace vanity throughput with the production SLO pack used by hosted inference teams: latency percentiles, goodput, and a monthly error budget.

Prompt to try

Write the SLO pack for a 7B-class chat model on one H100.

Definitions you must use (do not invent synonyms):

- TTFT: time to first streamed token after request accept.
- TPOT: mean inter-token latency after the first token.
- ITL: per-token inter-token latency histogram (not the mean).
- Goodput: tokens/s that landed inside the SLO, not raw tokens/s.
- Error budget: 1 - SLO_availability over 30 days; burn-rate alerts at 2% and 5% of budget per hour.

Numeric targets (interactive class):

- p50 TTFT <= 200 ms, p95 TTFT <= 800 ms, p99 TTFT <= 1.5 s
- p95 TPOT <= 30 ms
- availability 99.9% (successful 2xx that met the latency SLO)
- max queue wait 2 s then 429

Stages:

1. Write PromQL (or equivalent) for each target and the goodput recording rule.
2. Write the multi-window burn-rate policy (1h and 6h).
3. Design a load shape that can break p99 without breaking p50 (bursty prefills). Show why goodput falls while tokens/s stays flat.
4. State which metric you page on vs which you only dashboard.

Artifacts: slo.yaml, promql.md, burn_rate.md, load_shape.md, paging.md.

Grader contract

Goodput must be defined as in-SLO tokens, not raw decode rate. Burn-rate has two windows. A scenario where tokens/s is healthy and goodput is not is required. Paging on GPU util alone is a fail.

4. Multi-Tenant Isolation on Shared GPUs

Objective

Noisy-neighbor control: quotas, admission, and a hard isolation story (MIG, process placement, or rate-limit + KV partition).

Prompt to try

Three tenants share one 8xH100 node: A interactive (latency SLO from prompt 3), B batch (no TTFT SLO, 4-hour jobs), C eval (bursty, low duty).

Design isolation that satisfies:

- A cannot miss p95 TTFT because C started a 32k prefill.
- B cannot be starved for more than 10 minutes of GPU-seconds per hour.
- Weights may be shared (same base model) but KV pages and queues are per-tenant.
- A compromise of C's API key cannot read A's prompts or KV.

Stages:

1. Choose an isolation primitive: MIG instance, MPS+quota, or separate processes + cgroup + NVLink partition. State what it does and does NOT isolate (memory bandwidth? NVLink? GPU L2?).
2. Per-tenant: QPS cap, max_num_seqs, KV page budget, priority.
3. Write the admission algorithm when sum of budgets > physical pages (oversubscribe policy).
4. Threat model: tenant C sends a 1M-token prefill and a prompt that tries to dump another tenant's prefix cache. Show the reject path.
5. Metrics: llm_kv_cache_usage_ratio{tenant}, llm_ttft_seconds{tenant}, gpu_bytes_moved{tenant}.

Artifacts: isolation.md, quotas.yaml, admission.md, threat.md, metrics.md.

Grader contract

Must name what the chosen primitive does not isolate. KV and queues per-tenant. Cross-tenant prefix-cache read is a required reject. A's SLO named as a constraint on C's max prefill.

L3 Cache, adapters, and routing

5. Prefix / Radix Cache Across Replicas

Objective

Shared prefix caching (RadixAttention-style) with correctness under eviction and replica routing — the difference between a demo cache and a fleet cache.

Prompt to try

A chat product has 70% prefix overlap (system prompt + tool schema). Design a prefix cache that is correct and fleet-aware.

Invariants:

- I1 Safety: a cache hit may only return KV for a byte-identical prefix (hash the token ids, not the string).
- I2 Isolation: tenant A's prefix is never a hit for tenant B even if bytes match, unless an explicit allow-shared-system-prompt flag is on and the prefix is on an allowlist.
- I3 Eviction: LRU on nodes of the radix tree; evicting a parent invalidates children.
- I4 Routing: the gateway may stick a session to a replica that holds its prefix; if it cannot, it is a miss, not a cross-replica KV steal.

Stages:

1. Data structure: radix node = (token-id span, kv-block-refs, tenant, refcount).
2. Hashing: specify the hash (e.g. blake3 of token ids) and why UTF-8 string hash is illegal.
3. Gateway routing: consistent-hash on (tenant, conversation_id) with fallback. Show a 3-replica example when replica 2 dies mid-session.
4. Metrics: llm_prefix_cache_hit_ratio{tenant}, llm_prefix_cache_bytes, llm_routing_stick_fail_total.
5. Give one counterexample where a naive string-prefix cache would return wrong KV (tokenization edge).

Artifacts: cache.md, hash.md, routing.md, counterexample.md, metrics.md.

Grader contract

Token-id hash required; string hash is a fail. Cross-tenant hit forbidden unless allowlisted. Evict-parent-invalidates-children stated. Tokenization counterexample required.

6. Multi-LoRA Serving with Adapter Hotswap

Objective

One base model, many LoRA adapters, hot-load without dropping the base KV pool or mixing adapters on a batch step.

Prompt to try

Serve 40 LoRA adapters (rank 16, same target modules) on one base 8B instruct model.

Rules:

- A batch step may contain multiple adapters only if the kernel path supports it; otherwise homogeneous batches. State which you chose and why.
- Hotswap: loading adapter #41 must not evict base weights. Evict the LRU adapter that is not in R.
- Request header x-adapter-id selects the adapter; unknown id -> 404, not silent base model.
- Max resident adapters bounded by GPU leftover after base + KV pool. Show the memory equation.

Stages:

1. Memory equation: $\text{base} + N_{\text{res}} * \text{adapter} + \text{KV_pool} + \text{activation} \leq \text{GPU_mem} * \text{util}$.
2. Scheduler: how R is partitioned when 5 adapters are active and $\text{max_num_seqs}=32$.
3. Hotswap sequence (stop new admits for victim -> drain -> unload -> load -> ready).
4. Correctness test: two concurrent streams with adapters X and Y must not produce a token from the other's adapter (activation leak).
5. Metrics: `llm_adapter_resident`, `llm_adapter_swap_seconds`, `llm_adapter_miss_total`.

Artifacts: `memory.md`, `scheduler.md`, `hotswap.md`, `leak_test.md`, `metrics.md`.

Grader contract

Unknown adapter is 404 not base. Memory equation has all four terms. Activation-leak test described. Drain-before-unload in the swap sequence.

L4 Disaggregation and scale-out

7. Prefill/Decode Disaggregation

Objective

Split prefill and decode onto different pools (PD disaggregation) with an explicit KV handoff protocol and a tail-latency story.

Prompt to try

Design PD-disaggregated serving for a 70B TP=2 model.

Pools:

- Prefill pool: 4xH100, optimized for long prefills (chunked prefill allowed).
- Decode pool: 8xH100, optimized for large batch decode.

Handoff:

- After prefill, KV blocks move to a decode replica. Specify transport (RDMA / NVLink / GPU-direct TCP) and the unit (paged block, not full tensor dump if you can avoid it).
- Consistency: decode may not start until the exact block set for tokens [0, s) is present and checksummed.
- Failure: if handoff fails, retry once to another decode replica; then 504. Prefill result is not silently truncated.

Stages:

1. Sequence diagram: client -> gateway -> prefill -> handoff -> decode -> stream.
2. Chunked prefill: 2k-token chunks, how partial KV is committed.
3. Scheduling: a 32k prefill must not block decode slots on the decode pool.
4. SLO: p95 TTFT includes prefill + handoff. Budget the 800 ms p95 across the two pools.
5. Metrics: `llm_handoff_seconds`, `llm_handoff_fail_total`, `llm_kv_bytes_moved`, `llm_prefill_chunk_total`.

Artifacts: `sequence.md`, `transport.md`, `slo_budget.md`, `failure.md`, `metrics.md`.

Grader contract

TTFT budget split across pools. Checksum or equivalent on KV handoff. Decode pool does not run 32k prefills. Retry-once-then-504 specified.

8. Tensor/Pipeline Parallel Failover

Objective

A TP/PP group is one logical replica. Design restart, reshard, and in-flight request behavior when one rank dies.

Prompt to try

70B served TP=4 on four H100s in one NVLink domain. Rank 2 SIGTERM mid-decode.

Required behavior:

- In-flight streams: either complete on a reconstituted group or fail the request with 503 + retryable. No silent token drop.
- Restart: NCCL/RCCL group rebuild, weights reload from a local cache or sharded object store, dummy warm, ready.
- Gateway: stop sending to the group the instant rank 2 is unready. Existing connections drained or reset.
- Time budget: group ready again within 90 s if weights are local NVMe; state the number if they are remote.

Stages:

1. Failure detector (heartbeat vs NCCL timeout vs kube liveness). Pick one primary and say why NCCL timeout alone is insufficient.
2. Request fate table: prefill-not-started / prefill-in-flight / decode-in-flight / just-handoff'd.
3. Weight layout: how shards sit on disk so rank 2 can be replaced by a spare node with the same shard id.
4. Prove you do not double-emit tokens to the client on retry (idempotent decode cursor).
5. Metrics: `llm_replica_rebuild_seconds`, `llm_inflight_fail_total{phase}`, `nccl_timeout_total`.

Artifacts: `detector.md`, `fate_table.md`, `weights.md`, `idempotency.md`, `metrics.md`.

Grader contract

Fate table covers four phases. Idempotent cursor required. NCCL-timeout-only detector is a fail. Rebuild SLO stated for local vs remote weights.

L5 Control plane and safety

9. Inference Gateway: Auth, Quotas, Canary, Schema

Objective

The fleet front door: authn/z, per-key quotas, model routing, canary by weight, and request schema validation — Gateway API / Envoy AI Gateway / llm-d class.

Prompt to try

Write the gateway config model for 12 models, 200 API keys, two environments (canary 5%, stable 95%).

Must include:

- Auth: bearer key -> tenant + plan. Missing/invalid = 401, not 500.
- Schema: reject unknown fields in strict mode; cap `max_tokens` per plan.
- Routing: model alias (gpt-x) -> real deployment. Canary by header `x-canary` or by consistent hash of key at 5%.
- Quotas: token bucket per key (rpm and tpm). 429 with `Retry-After` and `X-RateLimit-*` headers.
- Shadow: 1% of stable traffic cloned to canary without affecting the client (fire-and-forget, drop response).
- Dual-SLO promotion: canary may become stable only if p95 TTFT and a quality probe (exact-match on a 20-prompt eval set) both hold for 45 minutes.

Stages:

1. Config objects: Key, Plan, ModelRoute, CanaryPolicy, Quota.
2. Request path sequence with every reject code.
3. Promotion state machine including automatic rollback on burn-rate.
4. What is NOT the gateway's job (KV cache, batching) — keep the boundary clean.
5. Metrics: `gateway_requests_total{route,code}`, `gateway_canary_quality_pass`, `gateway_shadow_total`.

Artifacts: `config.md`, `sequence.md`, `promote.md`, `boundary.md`, `metrics.md`.

Grader contract

401 vs 429 vs 400 distinguished. Shadow does not block client. Promotion needs BOTH latency and quality. Gateway must not claim to schedule KV.

10. Supply Chain, Signed Weights, Distroless Runtime

Objective

Air-gapped-capable inference image: signed weights, SBOM, non-root, no shell, reproducible pull.

Prompt to try

Produce the supply-chain contract for an inference node that may run air-gapped.

Requirements:

- Image: distroless or equivalent, USER non-root, no package manager, no sshd. Engine binary + tokenizer + config only.
- Weights: content-addressed (sha256). Admission webhook or node agent refuses to map a volume whose digest != pinned digest in the Model object.
- SBOM: image and weight artifact listed. Signing: cosign-style (keyless or hardware key); verify at pull and at ready.
- Reproducibility: given Model CR (name, digest, engine version, dtype, tp), two clusters produce bit-identical serving config.
- Rollback: previous digest remains on local cache until the new ready probe passes.

Stages:

1. Model CRD fields.
2. Admission sequence when a human tries to mount unsigned weights.

3. Ready probe includes signature + digest check, not just "GPU up".
4. Threat: a compromised build agent pushes a backdoored tokenizer. Where is it caught?
5. Artifacts list for an air-gap bundle (images, sigs, SBOMs, policy).

Artifacts: crd.md, admission.md, ready.md, threat.md, bundle.md.

Grader contract

Ready includes digest+sig, not only GPU. Unsigned mount denied. Tokenizer backdoor has a catch point. Non-root + no shell stated.

L6 Industry-hard combined systems

11. Heterogeneous Fleet Placement + Power Cap

Objective

Schedule replicas across H100/H200/B200/MI300 with different VRAM, interconnect, and a site power cap — the real capacity problem, not a YAML hobby.

Prompt to try

Cluster: 16xH100 80GB, 8xH200 141GB, 4xB200, 8xMI300X. Site power cap 1.4 MW. Models: 8B (TP=1), 70B (TP=2 or TP=4), 405B-class (TP=8 + PP=2).

Place replicas so that:

- Interactive 8B prefers lowest TTFT hardware.
- 70B interactive may sit on H200 to drop TP width.
- 405B only on parts that can form a working interconnect domain.
- Sum of estimated TDP at planned utilization $\leq$ 1.4 MW with 10% headroom.
- A B200 maintenance drain must not evict ALL 70B interactive capacity.

Stages:

1. Capability matrix: VRAM, peak HBM, interconnect domain, FP8/FP4 support, estimated TDP at 70% util.
2. Placement algorithm (bin packing with constraints), not "the scheduler will figure it out".
3. Power ledger: watts per replica type * count + host overhead.
4. Drain plan for one B200 tray.
5. Metrics: cluster_power_watts, cluster_allocatable_decode_tps{model}, placement_infeasible_total.

Artifacts: matrix.md, pack.md, power.csv, drain.md, metrics.md.

Grader contract

Power ledger is numeric and $\leq$ 1.26 MW planned (10% headroom under 1.4). Drain plan leaves some 70B interactive. Interconnect domain respected for 405B. No handwavy 'autoscaler decides'.

12. Full-Stack Correctness under Combined Optimizations

Objective

The hardest common production move: chunked prefill + prefix cache + speculative decoding + PD disaggregation + quantization, with a single correctness contract.

Prompt to try

Enable ALL of the following on one 70B deployment and keep answers correct:

- A. Chunked prefill (2k)
- B. Prefix cache with token-id hashing
- C. Speculative decoding (draft 1.5B, verify on 70B)
- D. PD disaggregation with checksummed KV handoff
- E. FP8 weights, KV in FP8 or FP16 (you choose, justify)

Correctness contract:

- C1 Greedy decode with temperature=0, seed unused, must match a reference single-GPU FP16 engine on a 25-prompt set after stripping only trailing whitespace. If you cannot match FP16, state the allowed numeric drift and WHY (FP8) and shrink the set to prompts where you still claim exact match.
- C2 Speculative accept/reject must not emit a token the verifier would not have emitted at that position.
- C3 Prefix-cache hit + chunked prefill + handoff must produce the same tokens as a cold prefill on the decode pool.
- C4 A failed speculator falls back to vanilla decode, no 5xx.

Stages:

1. Data-flow diagram with every optimization marked.
2. Speculative algorithm: draft k, verify, accept prefix, resample policy on rejection.

3. Combined test matrix: 2x2x2 of {cache hit, miss} x {short, long prefill} x {spec on, off} – expected relation to reference.
4. Kill switch: feature flags with a known-good path (all off except continuous batching).
5. Metrics: llm_spec_accept_ratio, llm_match_ref_total, llm_fallback_total{reason}.

Artifacts: flow.md, spec.md, matrix.md, flags.md, metrics.md.

Grader contract

C2 is non-negotiable (no extra tokens from the draft). C3 requires cache+chunk+handoff == cold path. Kill switch exists. If exact-match vs FP16 is abandoned, the drift bound is explicit and scoped. 'It should be close' is a fail.

13. Multi-Region Session Affinity and Disaster Replay

Objective

Two regions, session-sticky KV/prefix, region evacuate, and deterministic replay of a production trace for debugging — the incident-response endgame.

Prompt to try

Regions R1 and R2. Sessions prefer R1. Prefix cache and decode KV are region-local.

Incident: R1 loses its GPU subnet for 15 minutes.

Design:

- DNS/anycast or active-active gateway. New sessions go to R2 immediately.
- In-flight R1 streams: 503 retryable; client resumes with last received token index (byte-accurate).
- Prefix cache is NOT replicated live. State the hit-rate cliff and the warm-up policy on R2 (optional async seed of the system-prompt allowlist only).
- After R1 returns: drain back, no split-brain on conversation_id (one region owns a session at a time; fencing token).
- Replay: given a sampled trace (request body, adapter id, cache-hit bool, token-id prefix hash, decode cursor), a canary replica can reproduce the next 32 tokens in temp=0. If it cannot, the trace records which optimization made replay impossible.

Stages:

1. Ownership and fencing for conversation_id.
2. Evacuate timeline (t+0 detect, t+15s new sessions, t+60s in-flight declared dead).
3. Warm-up policy on R2 and the expected hit-ratio curve for 30 minutes.
4. Replay file schema and the list of features that break replay (must include speculative decoding unless you disable it on canary).
5. Metrics: region_failover_seconds, session_fence_conflict_total, replay_match_ratio.

Artifacts: fence.md, timeline.md, warmup.md, replay.schema.json, metrics.md.

Grader contract

Session fencing exists (no dual-owner). Resume by token index. Replay schema names features that break determinism. System-prompt seed is optional and allowlisted, not a live KV mirror fantasy.

L7 Kubernetes-native inference control plane (2026)

14. LLMInferenceService + InferencePool + Gateway API Inference Extension

Objective

Specify the 2026 K8s well-lit path: KServe LLMInferenceService as control plane, llm-d InferencePool as workers, Gateway API Inference Extension EndpointPicker for prefix- and load-aware routing. Round-robin is a fail.

Prompt to try

A platform team must serve a 70B-class model on 8 workers (TP=2) on Kubernetes. Use only these objects (name them correctly):

- LLMInferenceService – desired model, runtime=vLLM, disagg flags
- InferencePool – worker set, scales independently of the router
- InferenceModel / HTTPRoute – binds a model name to the pool
- Gateway + Gateway API Inference Extension EndpointPickerConfig
- scorers: prefix-cache-scorer, load-aware-scorer, queue-scorer
- LeaderWorkerSet if a multi-node TP group is one replica

Rules:

- A 100-token prompt and a 100,000-token prompt MUST NOT count as equal load to the picker.
- Prefix-cache-aware routing is on. State the metadata the EPP scrapes (KV usage, prefix hashes, running/

waiting).

- Autoscaling: KEDA on `llm_scheduler_waiting` and `llm_kv_cache_usage_ratio`, NOT on CPU. Scale-to-zero only for the batch class.
- Canary: 5% weight on a second InferencePool with a newer engine image.

Stages:

1. Object graph (router vs pool scale independently).
2. EPP decision for three requests: shared 8k system prefix, unique 32k RAG context, tiny 20-token chat.
3. EPP down: fail-open vs fail-closed. Defend against SLO + availability.
4. Metrics: `inference_picker_prefix_hit`, `inference_picker_queue_score`, `httproute_weight`.

Artifacts: `objects.yaml.md`, `epp.md`, `scale.md`, `failmode.md`, `metrics.md`.

Grader contract

Round-robin as the only policy is a fail. EPP features include KV and prefix, not only replica count. Waiting-queue and KV gauges drive scale, not CPU. Router and pool scale independently.

15. GPU Allocation: DRA vs MIG vs Time-Slicing vs Device Plugin

Objective

Place inference pods with DRA for NVLink topology, MIG for hard isolation, time-slicing only for best-effort eval.

Prompt to try

Cluster: 16xH100. Workloads: (A) 70B TP=2 interactive, (B) 8B multi-LoRA SaaS, (C) jittery eval.

Choose an allocation primitive per workload:

- A needs two GPUs in the SAME NVLink domain. `nvidia.com/gpu=2` is insufficient if the pair crosses the node.
- B may share a GPU only if isolation matches the tenant SLO. Time-slicing is illegal for A.
- C may use time-slicing or fractional sharing.

Stages:

1. Capability table: DRA topology, MIG, time-slice, classic device plugin – memory isolation? SM isolation? NVLink?
2. DRA ResourceClaim for A (two NVLink-connected GPUs).
3. Preemption via Kueue or PriorityClass so eval cannot steal A's pair.
4. One GPU in A's pair dies: rebind as a pair or fail the replica. Never silent TP=1 fallback.
5. Metrics: `dra_claim_pending`, `gpu_isolation_class{workload}`.

Artifacts: `table.md`, `claim.yaml.md`, `preempt.md`, `failure.md`, `metrics.md`.

Grader contract

A cannot be time-sliced. NVLink-domain constraint is explicit. Silent TP=1 fallback on pair loss is a fail. Table states what each primitive does NOT isolate.

16. Batch vs Service Scheduling: Kueue/KAI/Volcano vs HPA

Objective

Gang-schedule a TP replica and a batch job without putting interactive HTTP through the batch queue.

Prompt to try

Same cluster runs:

- Interactive InferencePool (KEDA on queue depth)
- Offline JSONL batch (10M prompts, throughput-only)
- A TP=8 replica that is useless unless all 8 ranks start together (gang)

Use Kueue or KAI/Volcano for gang + fair-share of batch and TP=8.

Do NOT send interactive HTTP through the batch queue.

Backfill batch only if it cannot raise interactive p95 TTFT above SLO.

Batch is preemptible; interactive gang is not.

Stages:

1. Queue topology and preemption edges.
2. Gang contract for TP=8: all-or-nothing, timeout, partial-pod cleanup.
3. Worked example: 6 GPUs free, batch wants 4, interactive scale-up wants 2, TP=8 waiting for 8. Determine admit/deny.
4. One paragraph: Slurm backfill as ancestor, then the K8s mapping.
5. Metrics: `kueue_admitted_workloads{queue}`, `gang_partial_timeout_total`, `interactive_slo_burn{queue}`.

Artifacts: `queues.md`, `gang.md`, `example.md`, `mapping.md`, `metrics.md`.

Grader contract

Interactive HTTP is not gang-queued. TP=8 is all-or-nothing. Worked example with 6 free GPUs is determinate. Partial gang leftovers cleaned up.

L8 Slurm, HPC hybrids, and SLA-driven factories

17. Slurm Batch Inference vs Kubernetes Serving

Objective

Clean contract: Slurm owns gang training and offline token jobs; Kubernetes owns latency SLOs. Crossing requires a digest-pinned handoff.

Prompt to try

Site has Slurm (GRES, fair-share, backfill, Pyxis/Enroot) and Kubernetes (vLLM, Gateway API).

Place:

1. Chat API, p95 TTFT 800 ms
2. Nightly eval of 50k prompts
3. Multi-node 405B training
4. RAG embed burst (30 min, no SLO)
5. Canary of a new engine image

Rules:

- Live HTTP SLO cannot sit in an sbatch queue.
- Exclusive 64+ GPU topology jobs should not be a naked Deployment.
- Handoff: OCI digest from Slurm training -> Model CR on K8s, pinned.

Stages:

1. Placement table with one-line why.
2. Slurm partition + QOS for (2) and (4).
3. K8s InferencePool + canary for (1) and (5).
4. Prevent a researcher salloc on the inference node group.
5. Metrics: slurm_pending_gpus, k8s_allocatable_gpus, leaked_gpu_seconds.

Artifacts: placement.md, slurm.conf.md, k8s.md, isolation.md, metrics.md.

Grader contract

Chat API is not sbatch. 405B training is not a naked Deployment. Digest-pinned handoff exists. Inference partition cannot be salloc'd by researchers.

18. Hybrid Control Plane: Slinky / K8s-fronts-Slurm / Side-by-side

Objective

Pick one hybrid and make GPU ownership exclusive. Dual-schedule of the same GPU must be structurally impossible.

Prompt to try

Researchers want sbatch; product wants LLMInferenceService. Pick exactly one:

- H1 Slinky: slurmctld/slurmd as pods; Slurm schedules GPUs K8s also sees.
H2 K8s-fronts-Slurm: API on K8s, each replica is a Slurm job running vLLM. Scale-out = sbatch.
H3 Side-by-side: disjoint node groups; only a weight registry in common.

For the chosen H:

- Who is source of truth that GPU 3 is free?
- How is double-schedule of GPU 3 made impossible?
- Ready probe if the replica is a Slurm step?
- Burst 0->32 decode GPUs in 10 minutes: can this H hit it?

Stages:

1. Ownership invariant (one scheduler per GPU).
2. Double-schedule prevention (taints, exclusive GRES, device-plugin filter, or disjoint nodes).
3. Ready/live mapping for a Slurm-backed replica.
4. Burst math: queue delay + pull + load + dummy warm.
5. Metrics: dual_schedule_conflict_total (must stay 0), burst_gpus_per_min.

Artifacts: choice.md, ownership.md, ready.md, burst.md, metrics.md.

Grader contract

Exactly one owner per GPU. Dual-schedule is impossible by construction. Ready probe defined. Burst number is computed.

19. SLA-Driven Graph Deployment (Dynamo / llm-d recipe)

Objective

Declare TTFT and ITL; derive prefill vs decode pool sizes. Autoscaler reacts to which SLO is burning.

Prompt to try

Object (DynamoGraphDeployment-style or llm-d recipe):

```
spec.model: 70B-class
spec.backend: vLLM | SGLang | TRT-LLM
spec.sla.ttft_ms: 200
spec.sla.itl_ms: 20
spec.autoApply: true
```

Derive:

- N_prefill, N_decode, TP width, PD on/off
- Why 200 ms TTFT forces or forbids disagg at given ISL/OSL
- If TTFT burns and ITL is fine: add prefill, not decode

Stages:

1. Worked sizing: ISL=2k, OSL=256, 40 rps, H100. Show N_prefill and N_decode.
2. When autoApply stays aggregated (short prompts, no RDMA).
3. Multimodal: add Encode pool (EPD); when it pays vs colocate-encoder.
4. Flap kill switch (min/max, cooldown).
5. Metrics: sla_ttft_error, sla_itl_error, derived_pool_size{role}.

Artifacts: sizing.md, policy.md, epd.md, hysteresis.md, metrics.md.

Grader contract

Sizing is numeric for ISL/OSL/rps. TTFT burn scales prefill. EPD has a pays-off rule. Cooldown exists.

L9 Novel data paths: hierarchical KV, EPD, heterogeneous PD

20. Hierarchical KV (Mooncake / LMCache / fabric memory)

Objective

HBM to DRAM to NVMe to remote pool, with a hitch budget so decode never waits on storage for the next token of an admitted session.

Prompt to try

Agent sessions keep 8k-32k KV across turns. HBM cannot hold all of them.

Tiers: T0 HBM, T1 host DRAM, T2 local NVMe, T3 remote RDMA pool.

Invariants:

- Decode never blocks on T2/T3 for the next token of an admitted session. If T0 is full, swap a cold session; do not hitch the hot one.
- Dual-path load allowed: storage->prefill OR storage->decode->RDMA->prefill when prefill NICs saturate.
- Hash is token-id + tenant.

Stages:

1. Which sessions live on which tier (recency, remaining max_tokens, tenant class).
2. Hitch budget in milliseconds; shed if exceeded.
3. Dual-path trigger (prefill NIC saturation).
4. Break-even hit rate: T3 vs buying more T0 GPU.
5. Metrics: kv_tier_hits{tier}, kv_hitch_ms, kv_shed_total.

Artifacts: tiers.md, hitch.md, dualpath.md, breakeven.md, metrics.md.

Grader contract

Hot decode cannot hitch on NVMe. Dual-path trigger is concrete. Break-even is a number. Token-id+tenant hash reused.

21. Encode-Prefill-Decode + Heterogeneous Accelerators

Objective

Vision encode on cheaper GPUs, prefill on compute-rich, decode on bandwidth-rich; embeddings and KV cross mixed interconnects and dtypes.

Prompt to try

Traffic is 40% image+text. Hardware: L40S encode, H100 prefill, H200 decode.

EPD rules:

- Embedding cache keyed by content hash of media bytes, not filename.
- Router hashes media + text prefix for stickiness.
- Dtype per hop: encode FP16, prefill FP8, KV transfer format stated. No silent NaNs.
- No GPUDirect between L40S and H200. Name the transport and when EPD is a net loss.

Stages:

1. Topology and when to colocate encoder with prefill.
2. Embedding-cache key and eviction.
3. Dtype contract at each hop.
4. Re-budget 800 ms p95 TTFT including encode.
5. Metrics: encode_ms, embed_cache_hit, epd_net_loss_total.

Artifacts: topo.md, embed.md, dtypes.md, slo.md, metrics.md.

Grader contract

Media hash is content, not path. TTFT includes encode. Net-loss condition stated. Dtype at each hop explicit.

Scoring the ladder

Report a vector, not a single percentage: pass/fail per prompt, plus whether required artifacts exist. Suggested headline: highest L completed with all graders in that L green. L1-L3 is table stakes. L6 combined optimizations, L7 EPP-not-round-robin, L8 exclusive GPU ownership, and L9 hitch budget are the distinguishing items.

Automatic suite fails: inventing a metric name instead of TTFT/TPOT/goodput; collapsing live and ready; string-hashing prefixes; serving an unknown LoRA as the base model; gateway that claims to manage KV pages; power plan that exceeds the cap; speculative decoding that can emit an unverified token.

Optional execution harness: implement L1–L2 against vLLM or SGLang on one GPU and attach real Prometheus scrapes. L3–L6 may stay as specified systems with worked examples if hardware is unavailable; the grader still requires the worked occupancy/power/test matrices.

Companion to the Long-Horizon Multi-Agent Orchestration suite: that suite tests obligation-preserving agency; this suite tests whether the obligations themselves match production inference standards.